

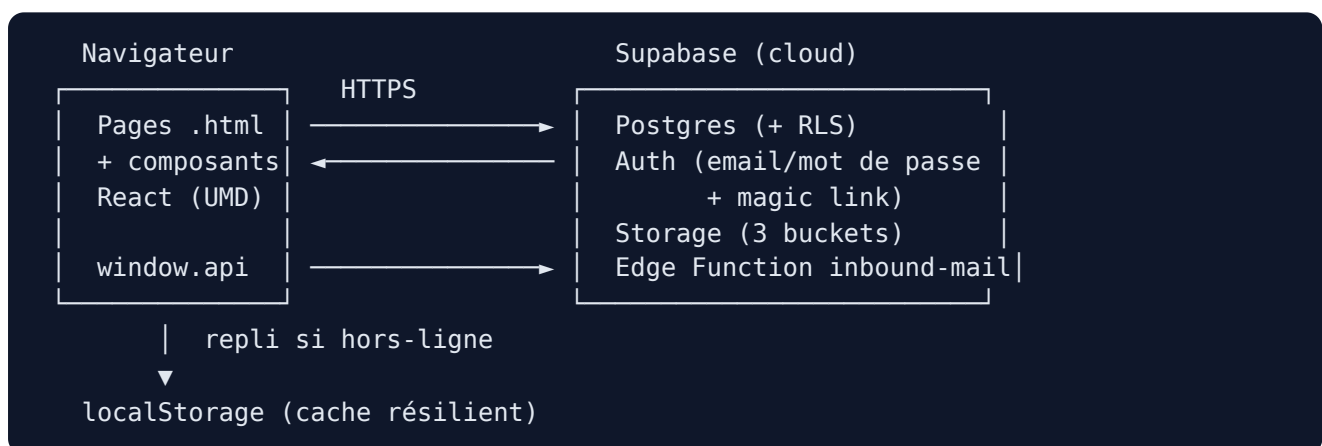
Hub Astorya — Comment ça fonctionne

Document explicatif du fonctionnement du Hub : architecture, circulation des données, modules et workflows. Pour **déployer/migrer**, voir [DEPLOIEMENT.md](#) . Pour le **détail contractuel/commercial**, voir [Hub_Astorya_Documentation_Complete.md](#) .

1. En une phrase

Le Hub Astorya est un **CRM/ERP web** pour ASTORYA SGI : il suit les prospects et clients, le pipeline commercial, produit les pièces commerciales (devis → commande → bon de livraison → facture, + acomptes et avoirs), gère les projets, le support/hotline, le stock, et consolide la trésorerie — le tout dans une interface unique adossée à Supabase.

2. Architecture en clair



- **Front 100 % statique** : chaque écran est un fichier `.html` qui charge des composants React en **UMD** (pas de framework de routing, pas de serveur Node). React et Supabase sont chargés depuis des CDN ; les composants depuis `components/` et `dist/components/` .
- **Pas de build côté serveur** : les composants JSX (`components/*.jsx`) sont compilés une fois en JS (`dist/components/*.js`) via Babel (`cd tools && npm run build`). Le `dist/` est **versionné** et servi tel quel.
- **Couche d'accès données unique** : `components/api.js` expose `window.api` (modules `clients` , `opportunities` , `contacts` , `actions` , `commercialDocs` , `projects` , `deliveryNotes` , `tickets` , `notifications` , `inboundRequests` , `emailTemplates` , `auth` ...). Tous les écrans passent par là — aucune requête Supabase éparpillée.
- **Cache-buster** : chaque inclusion de script porte `?v=AAAAAMJJHHMM` . On l'incrmente à chaque livraison pour forcer les navigateurs à recharger les fichiers modifiés (sinon ils servent une vieille version en cache).

Le pattern de résilience (important)

Quand Supabase est joignable, tout est lu/écrit en base. Si une écriture échoue (réseau coupé, table absente, RLS), `api.js` retombe sur `localStorage` et prévient l'utilisateur (toast « Enregistrement local »). À l'inverse, les lectures fusionnent base + `localStorage`. Avantage : l'app ne « plante » jamais. Limite : des données peuvent rester locales tant que la base n'est pas revenue — d'où l'importance que le schéma soit complet en base (cf. §3).

3. Données & persistance

- **Tables « cœur »** : `clients`, `contacts`, `opportunities`, `actions`, `contracts`, `tickets`, `profiles`, `groups` ... (schéma reconstruit dans `sql/SCHEMA_COMPLET.sql`).
- **Tables « commerciales »** : `commercial_docs` (devis/commande/BL/facture/facture_acompte/avoir), `commercial_doc_lines`, `commercial_articles`, `commercial_company_settings`, compteurs de numérotation...
- **Tables « projets/livraison »** : `projects`, `project_items`, `delivery_notes`, `delivery_note_items`, `project_events`.

Pattern « colonnes réservées + `data jsonb` »

La plupart des tables ont quelques colonnes Postgres « dures » (celles qu'on filtre trie : `id`, `client_id`, `status`, `total_ttc`, `doc_date` ...) **plus** une colonne `data` de type `jsonb` qui absorbe tout le reste (champs variables : SIREN, SIRET, NAF, paiements, métadonnées d'acompte/avoir...). Cela évite une migration SQL à chaque nouveau champ. Exemple : sur un client, `name` / `city` sont des colonnes, mais `siren`, `siret`, `effectif`, `contacts_additionnels` vivent dans `clients.data`.

Numérotation des pièces

Les numéros (DEV-2026-0001, FAC-2026-0007, BL-2026-0003...) sont attribués par une fonction SQL atomique `commercial_next_doc_number(type)`, avec une contrainte `UNIQUE(type, année, séquence)` qui interdit tout doublon de numéro légal.

Sécurité d'accès (RLS)

Chaque table a le **Row Level Security** activé avec une policy « utilisateur authentifié requis ». Le modèle est volontairement **mono-société** : toute l'équipe ASTORYA voit toutes les données (pas d'isolation par client). Les accès `anon` (non connecté) sont révoqués.

4. Connexion & rôles

- **Connexion** (`/login`) : email + mot de passe, **ou** lien magique (OTP par email). Réinitialisation de mot de passe par email.

- **Garde d'authentification** : chaque page vérifie la session au chargement et masque son contenu jusqu'à confirmation (`auth-guard.js`) ; sans session → redirection `/login` .
- **Rôles & groupes** : `profiles` + `groups` + `profile_groups` . Un groupe porte une liste de modules autorisés (`access`), ce qui pilote les tuiles visibles sur l'accueil. (NB : ce filtrage est côté interface ; la sécurité de fond repose sur la RLS Supabase.)

5. Les modules

✓ = fonctionnel · 🚧 = page « Bientôt » (placeholder roadmap, non actif)

5.1 Accueil ✓ (`/` , ERPHome)

Grille de tuiles par module (filtrées selon le groupe de l'utilisateur), recherche globale (clients/opportunités), cloche de notifications, menu profil.

5.2 CRM — Pipeline ✓ (`/crm`)

Pipeline d'opportunités en **Kanban / Liste / Planning**, KPI (pipeline pondéré, taux de closing...), stades SPANCO (qualif → découverte → proposition → négo → gagné/perdu). Création d'opportunité, glisser-déposer entre stades.

5.3 Prospects & Clients ✓ (`/nouveau-prospect` , `/fiche-client`)

- **Création de fiche** : enrichissement automatique via **Pappers** (SIREN/SIRET → raison sociale, NAF, dirigeants, établissements) et **BODACC** (procédures collectives). Champs **SIREN et SIRET**.
- **Fiche client** : coordonnées, contacts (un principal + additionnels), opportunités liées, contrats, timeline d'actions, lancement d'appel 3CX.

5.4 Gestion commerciale ✓ (`/gestion-commerciale`) — cœur ERP, modèle Sage 50

Chaîne complète : **Devis** → **Commande** → **Bon de livraison** → **Facture**, chaque pièce héritant des lignes de la précédente (`parent_doc_id`).

- **Facture d'acompte** : depuis une commande ou un BL, on enregistre l'acompte réglé par le client (ex. 40 %). Crée une facture d'acompte **verrouillée**, dans sa propre catégorie, avec la **TVA ventilée par taux**.
- **Déduction automatique** : à la facturation finale, l'acompte déjà facturé est déduit (lignes négatives), de façon **idempotente** (jamais déduit deux fois).
- **Avoir** : toujours rattaché à une facture (total ou partiel), en négatif, déduit du solde dû.
- **Règlements** : enregistrement des paiements (montant, date, mode, réf.) ; calcul du « reste à payer » / « soldé ».

- **PDF** : génération pdfmake (devis, BL sans prix, factures...), envoi par email, téléchargement — chaque envoi est journalisé.

5.5 Trésorerie ([/tresorerie](#))

Vue consolidée des **encaissements** (règlements reçus, acomptes encaissés), des **avoirs émis**, et des **restes à recouvrer** par facture, avec navigation mois par mois et KPI. Alimentée automatiquement par les pièces commerciales.

5.6 Projets & Livrables ([/projets](#) , [/projet](#))

Vues Kanban / Tableau / Gantt / Calendrier. Une **commande** crée automatiquement un projet (colonne « Reçu »). Génération de **bons de livraison** avec **signature manuscrite** : à la signature, les articles livrés passent en « livré » et un évènement + notification sont créés.

5.7 Demandes entrantes ([/demandes-entrantes](#)) — automatisé email

Une boîte mail surveillée (via l'Edge Function [inbound-mail](#)) reçoit les demandes ; l'IA (Anthropic) extrait l'objet/produits/urgence, **rapproche le client **connu**, crée une « demande de devis à faire »** et notifie. L'écran liste les demandes en attente, permet de rattacher un client et de basculer en création de devis.

5.8 Ticketing / Hotline ([/ticketing](#))

Tickets support avec catégories, priorités, SLA, escalade vers un groupe. Intégration **3CX** : popup d'appel entrant, journal d'appels ([calls](#)), transcriptions ([call_transcripts](#)), commentaires.

5.9 Stock ([/stock](#))

Catalogue d'articles et parc d'**assets** (numéros de série, statuts : disponible/réservé/affecté/SAV/HS), compteurs agrégés par article.

5.10 Intelligence concurrentielle ([/intelligence-concurrentielle](#) ,

[/fins-contrats-concurrents](#)) Radar des fins de contrats concurrents (leasing/garanties), battle cards.

5.11 Autres écrans fonctionnels

- **Équipe commerciale** ([/equipe-commerciale](#)) · **Planning commercial** ([/planning-commercial](#)) · **Temps & Activités** ([/temps-activites](#) , sessions & évènements utilisateurs) · **Templates email** ([/templates-email](#)) · **Nouveau contrat** ([/nouveau-contrat](#) , PDF de contrat de services) · **Administration** ([/administration-utilisateurs](#) , utilisateurs/groupes/intégrations 3CX & Pappers).

5.12 Modules à venir

[/comptabilite](#) , [/facturation](#) , [/marketing](#) , [/rapports](#) , [/ressources-humaines](#) affichent une page « Bientôt » avec leur roadmap — ils ne sont pas encore actifs.

6. Les automatisations

Déclencheur	Effet
Email reçu sur la boîte surveillée	Edge Function → extraction IA → demande de devis + notification
Création d'une commande	Création/synchro d'un projet (+ commande d'achat fournisseur)
Création d'un BL	Génération du PDF (sans prix) attaché au projet
Signature d'un BL	Articles → « livré » + évènement projet + notification
Acompte réglé	Facture d'acompte verrouillée + déduction à la facture finale

7. Intégrations externes

- **Pappers** — enrichissement entreprise (SIREN/SIRET).
- **BODACC** — procédures collectives.
- **Anthropic (Claude)** — extraction des demandes entrantes (côté Edge Function).
- **3CX** — téléphonie (popup appel, journal, transcriptions).
- **Microsoft Teams** — notifications sortantes (webhook).
- **monday.com** — fournisseurs (intégration).

Les clés publiques (URL + clé *publishable* Supabase) sont dans `components/supabase-config.js`. Les secrets serveur (Anthropic, jeton webhook, service_role) vivent **uniquement** côté Edge Functions Supabase.

8. Sécurité (résumé)

- RLS activée partout, accès `anon` révoqués, modèle mono-société.
- En-têtes de sécurité + CSP servis par `vercel.json` (HSTS, X-Frame-Options DENY, nosniff, COOP, `frame-ancestors none` ...).
- Edge Function `inbound-mail` fermée par défaut (refuse si jeton non configuré), validation/bornage des entrées.
- Garde d'authentification anti-flash sur toutes les pages.
- Liens de notification assainis (pas de `javascript:`).

9. Comment le Hub évolue (cycle de modif)

1. Éditer un composant `components/*.jsx` (ne jamais éditer `dist/` à la main).
2. `cd tools && npm run build` → recompile vers `dist/components/*.js`.
3. Incrémenter le cache-buster `?v=...` dans les `.html`.

4. Commit + push. Le front statique est redéployé (Vercel ou autre).
 5. Changement de schéma → ajouter une **migration SQL idempotente** dans `sql/` et l'exécuter dans Supabase (et la documenter dans `sql/README.md`).
-

10. Pour aller plus loin

- Déploiement / migration serveur → `DEPLOIEMENT.md`
- Schéma de base → `sql/SCHEMA_COMPLET.sql` + `sql/README.md`
- Doc commerciale & contractuelle complète → `Hub_Astorya_Documentation_Complete.md`
- Conventions de développement → `DEVELOPMENT.md`
- Automatisation mail pas-à-pas → `GUIDE_Demandes_entrantes_Gmail.md`